SQL Query Generator

Project Description

The *SQL Query Generator* is an intelligent system that converts natural language questions into SQL queries. The main objective of this project is to simplify database interaction for users who do not have knowledge of SQL. Instead of writing complex SQL statements manually, users can simply enter their questions in plain English to retrieve the required data from the database.

The system is developed using Python and uses libraries such as Transformers, SQLite, and Gradio. A pre-trained language model (*distilgpt2*) is used to support natural language processing. The system analyzes the user input and identifies keywords related to employee information such as city, salary, or employee records.

Based on the detected keywords, the program automatically generates the appropriate SQL query using predefined conditions. These queries are executed on a SQLite database that contains employee data such as employee ID, name, salary, and city. The system then retrieves the matching records from the database.

A simple and interactive user interface is created using the Gradio framework. Through this interface, users can enter their questions, generate SQL queries, and view the results instantly. The interface also displays the generated SQL statement along with the retrieved database output.

The SQL Query Generator helps users access database information easily without requiring technical expertise in SQL. This system improves usability and demonstrates how natural language processing can be used to simplify database querying tasks.

Scenario 1: Employee Data Retrieval

In this scenario, a user wants to view all employee records stored in the database. Instead of writing a SQL query manually, the user enters a natural language request such as “Show all employees.” The system processes the request and identifies the employees table from the database.

The system analyzes the keywords in the request and automatically selects relevant columns such as id, name, salary, and city. Based on this analysis, the system generates the appropriate SQL query:

Example Query:

SELECT * FROM employees;

The generated query is executed on the SQLite database, and the employee records are retrieved successfully. The results are then displayed to the user through the Gradio interface. This scenario demonstrates how the system simplifies database access by converting natural language input into SQL queries.

Scenario 2: Employee Salary Filtering

In this scenario, a user wants to find employees whose salary is greater than a certain value. The user enters a request such as “Show employees with salary greater than 50000.”

The system analyzes the request and detects two important components:

The employees table

A salary condition greater than 50000

Using this information, the system generates the following SQL query:

Example Query:

SELECT * FROM employees WHERE salary > 50000;

The system executes the query and retrieves the matching employee records from the database. This scenario demonstrates how the SQL Query Generator can automatically create filtering conditions using the WHERE clause based on user input.

Scenario 3: Employee Location Filtering

In this scenario, the user wants to find employees working in a specific city. The user enters a request such as “Show employees from Chennai.”

The system analyzes the request and identifies the city column as the filtering condition. Based on this information, the system generates the SQL query:

Example Query:

SELECT * FROM employees WHERE city='Chennai';

After generating the query, the system retrieves the matching employee records from the database and displays them to the user. This feature helps users quickly filter records based on location without manually writing SQL queries.

Scenario 4: Security Validation

Security validation is an important part of the SQL Query Generator system. In this scenario, a user attempts to enter a malicious query that contains unsafe SQL commands such as DROP, DELETE, or ALTER.The system includes validation mechanisms that detect dangerous SQL keywords and suspicious patterns before executing the query. When such keywords are detected, the system blocks the query execution immediately and displays an appropriate error message to the user.This security feature prevents unauthorized or harmful operations on the database and ensures that the system remains safe and reliable. By filtering unsafe commands, the SQL Query Generator protects the database from potential security threats

Architecture Overview

The SQL Query Generator is designed as a secure, modular platform that integrates a Python backend with the IBM Granite 3.3 2B Instruct AI model for intelligent natural language processing. Its architecture emphasizes security, accuracy, and usability by combining query validation, AI-powered input parsing, and professional reporting.

The system workflow is SQL-centric: user input in natural language is captured through a conversational interface, processed by the AI model to detect relevant tables, columns, and conditions, and converted into safe and optimized SQLite SQL queries. The results can then be retrieved from the database and exported for documentation or reporting. This modular design allows independent management of the AI parser, database layer, and reporting tools, ensuring reliability and scalability.

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Transformers: Install via pip (pip install transformers)
- PyTorch: Install via pip (pip install torch)
- ReportLab: Install via pip (pip install reportlab)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites (Short)

Python Basics – Functions, lists, dictionaries for dynamic query and result handling.

SQL Fundamentals – SELECT, WHERE, JOIN, basic query structure.

Gradio – User input → Python → SQL → Result display.

Regex – Text pattern matching, input validation.

Database Concepts – Tables, columns, relationships, data types.

Security – SQL injection prevention, sanitize inputs.

Hardware Requirements

- Hardware Requirements (Optimized for Project)
- Processor – Intel i3 / AMD Ryzen 3 or equivalent (local development; Colab cloud handles heavy computation).
- RAM – Minimum 4 GB (8 GB recommended for smooth performance).
- Storage – Minimum 2 GB free for Python packages, Gradio dependencies, and exports.
- Display – 1280×720 resolution minimum for clear interface display.

Project Workflow

Phase 1: Environment Setup

Activity 1: Installing Required Libraries

In this activity, the necessary Python libraries such as Transformers, Torch, SQLite, and Gradio are installed. These libraries support natural language processing, database connectivity, and user interface development.

Activity 2: Project Environment Preparation

The Python environment is prepared to run the project code. This ensures that all dependencies and tools required for the SQL Query Generator are properly configured.

Phase 2: Database Development

Activity 1: Database Creation

In this activity, a SQLite database named *company.db* is created using Python. This database will store the employee information used in the project.

Activity 2: Table Creation and Data Insertion

A table called *employees* is created with fields such as employee ID, name, salary, and city. Sample employee records are inserted into the table to simulate real database data.

Phase 3: Model Initialization

Activity 1: Loading the NLP Model

In this activity, the pre-trained language model *distilgpt2* is loaded using the Transformers library. The model helps process natural language input and supports the query generation process.

Phase 4: Natural Language Query Processing

Activity 1: User Input Collection

The system collects the user's question through the interface. The user can enter queries related to employee data using natural language.

Activity 2: Input Processing

The user question is processed by converting it into lowercase and identifying important keywords such as employee, city, or salary conditions.

Phase 5: SQL Query Generation

Activity 1: Mapping Natural Language to SQL

The system analyzes the keywords present in the question and maps them to predefined SQL query conditions.

Activity 2: SQL Statement Creation

Based on the detected condition, the system automatically generates the corresponding SQL query.

Phase 6: Query Execution

Activity 1: Database Connection

The system connects to the SQLite database (*company.db*) using Python.

Activity 2: Executing the SQL Query

The generated SQL query is executed using the database cursor, and the matching records are retrieved from the employees table.

Phase 7: Result Display

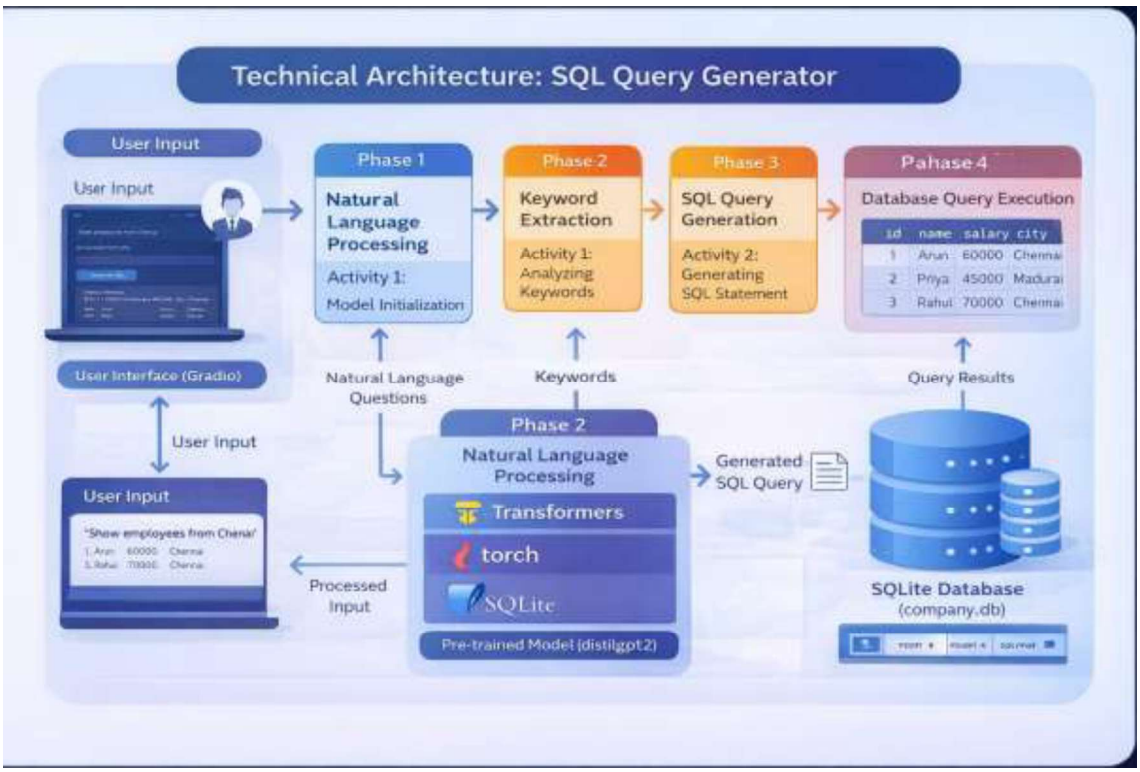
Activity 1: Displaying Query and Results

The generated SQL query and the retrieved results are displayed to the user through the Gradio interface.

Activity 2: User Interaction

The interface allows users to easily enter new questions and generate SQL queries without requiring knowledge of SQL syntax.

Technical Architecture



Database Schema

Table : employees

employees

id : INTEGER (Primary Key)

name : TEXT

salary : INTEGER

city :TEXT

Description

Field Name	Data Type	Description
id	INTEGER	Unique ID for each employee
name	TEXT	Name of the employee
salary	INTEGER	Salary of the employee
city	TEXT	City where the employee works

MILESTONE 1 : Environment Setup

Cell 1

Installed required libraries such as Transformers, Torch

```
!pip install transformers torch sqlite-web
```

In this milestone, the required libraries and dependencies for the project were installed. Libraries such as transformers and torch were installed using pip. This step prepared the development environment for implementing the SQL Query Generator system.

MILESTONE 2 : Database Creation

Cell 2

Created SQLite database *company.db* and employees table with sample data.

```python
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE employees(
id INTEGER PRIMARY KEY,
name TEXT,
salary INTEGER,
city TEXT
)
""")

cursor.execute("INSERT INTO employees VALUES (1,'Arun',60000,'Chennai')")
cursor.execute("INSERT INTO employees VALUES (2,'Priya',45000,'Madurai')")
cursor.execute("INSERT INTO employees VALUES (3,'Rahul',70000,'Chennai')")
cursor.execute("INSERT INTO employees VALUES (4,'Anu',30000,'Trichy')")

conn.commit()
conn.close()

print("Database created")
```

```
Database created
```

MILESTONE 3 : Model Initialization

Cell 3

Loaded the *distilgpt2* model using the Transformers library

```
from transformers import pipeline

generator = pipeline("text-generation", model="distilgpt2")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Googl
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json:   0%|          | 0.00/762 [00:00<?, ?B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
model.safetensors:   0%|          | 0.00/353M [00:00<?, ?B/s]
Loading weights:   0%|          | 0/76 [00:00<?, ?it/s]
GPT2LMHeadModel LOAD REPORT from: distilgpt2
Key                                     | Status     |  |
----------------------------------------+------------+--+-
transformer.h.{0, 1, 2, 3, 4, 5}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json:   0%|          | 0.00/124 [00:00<?, ?B/s]
tokenizer_config.json:   0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

MILESTONE 4 : Natural Language to SQL Conversion

Cell 4

Implemented *generate_sql()* function to convert natural

```python
def generate_sql(question):

    question = question.lower()

    if "all employees" in question:
        return "SELECT * FROM employees"

    elif "salary greater than 50000" in question:
        return "SELECT * FROM employees WHERE salary > 50000"

    elif "employees from chennai" in question:
        return "SELECT * FROM employees WHERE city='Chennai'"

    elif "employees from madurai" in question:
        return "SELECT * FROM employees WHERE city='Madurai'"

    else:
        return None
```

MILESTONE 5 : Query Execution

Cell 5

Executed generated SQL queries on the SQLite database and retrieved results.

```python
import sqlite3

question = input("Enter your question: ")

query = generate_sql(question)

print("Generated SQL:", query)

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

cursor.execute(query)

rows = cursor.fetchall()

for r in rows:
    print(r)

conn.close()
```

```
Enter your question: all employees
Generated SQL: SELECT * FROM employees
(1, 'Arun', 60000, 'Chennai')
(2, 'Priya', 45000, 'Madurai')
(3, 'Rahul', 70000, 'Chennai')
(4, 'Anu', 30000, 'Trichy')
```

MILESTONE 6 : User Interface Development

Cell 6

Developed a user interface using *Gradio* to input questions and display results.

```
!pip install gradio
```

Cell 7

Developed a user interface using *Gradio* to input questions and display results.

```
import gradio as gr
import sqlite3

def run_query(question):

    query = generate_sql(question)

    conn = sqlite3.connect("company.db")
    cursor = conn.cursor()

    cursor.execute(query)
    rows = cursor.fetchall()

    conn.close()

    result = f"Generated SQL:\n{query}\n\nResult:\n{rows}"
    return result


css = """
body {background-color:#0f172a;}
.gradio-container {background-color:#0f172a; color:white}
textarea {font-size:18px !important}
"""

with gr.Blocks(css=css) as demo:

    gr.Markdown("# SQL Query Generator")

    question = gr.Textbox(
        label="Enter your Question",
```

MILESTONE 7: User Interface Development

Cell 7 continue

Developed a user interface using *Gradio* to input questions and display results

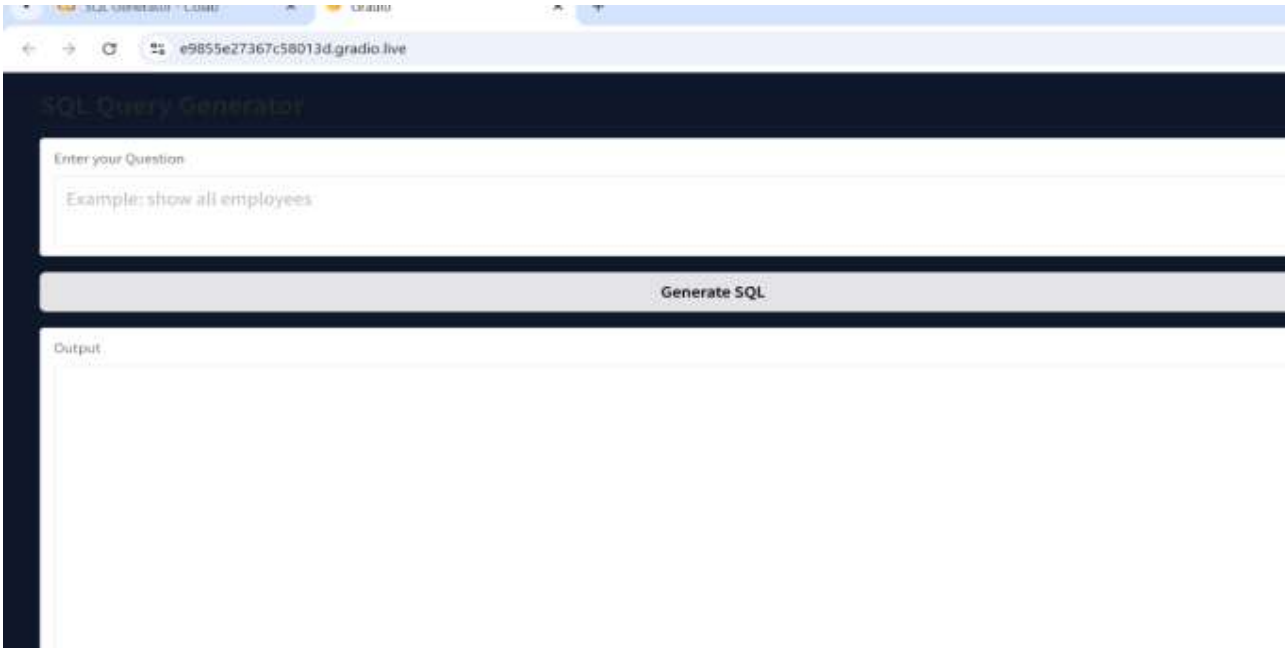
```
        label= Enter your Question ,
        placeholder="Example: show all employees",
        lines=2
    )

    btn = gr.Button("Generate SQL")

    output = gr.Textbox(
        label="Output",
        lines=12
    )

    btn.click(run_query, inputs=question, outputs=output)

demo.launch()
```

MILESTONE 8 : Testing and Development

Cell :

Access Points:

Public URL : https://585f20d8ec3644e1d2.gradio.live/



Test Case 1 :

Input:

salary greater than 50000

Test Case 2:

Input:

employees from chennai

Conclusion :

The SQL-based Query Generator system simplifies the process of writing database queries by converting natural language inputs into accurate SQL statements. It helps users who have limited knowledge of SQL to easily retrieve data from databases without manually writing complex queries. The system improves productivity, reduces errors, and ensures secure query generation through input validation and SQL injection prevention. Overall, this project demonstrates how SQL and Python-based technologies can be effectively combined to create a user-friendly and efficient database query solution.